

Script - Bandit 12 Decompression Loop

Two implementations of the same task: unwrap a repeatedly compressed file until plain text is reached. Written **after** solving [Bandit - Level 12](#) by hand — automating a process not yet understood produces a script that can't be debugged.

Concepts: [Linux - Nested Archives and Compression Layers](#)

Line-by-line syntax explanation: [Script - Code Walkthrough \(Bash & Python\)](#)

The Problem Shape

The task cannot be a fixed sequence of commands, because **each layer's type determines the next step's tool**, and that type is unknown until the previous layer is removed. That makes it a loop with a branch:

```
identify → decompress → identify → ... until plain text
```

Three properties drove every design decision:

1. **Output filenames are unpredictable** — `tar` extracts under the name stored inside the archive, `bunzip2` invents `.out` when it can't strip a suffix. Nothing can be assumed; it has to be discovered.
 2. `gunzip` / `bunzip2` **consume their input**, `tar` **doesn't** — inconsistent state after each iteration.
 3. **The loop needs a hard stop**. If a layer is misidentified and the data stops changing, an unbounded loop runs forever. Anything driven by external data needs an escape hatch.
-

Bash version — `unwrap.sh`

Full source

```
#!/usr/bin/env bash
#
# unwrap.sh - Recursively unwrap a repeatedly compressed file.
# Written for OverTheWire Bandit Level 12, but works on any nested archive.
#
# Usage: ./unwrap.sh <file> [max_layers]
#
set -uo pipefail

readonly INPUT="${1:-}"
readonly MAX_LAYERS="${2:-30}"

if [[ -z "$INPUT" || ! -f "$INPUT" ]]; then
    echo "Usage: $0 <file> [max_layers]" >&2
    exit 1
fi
```

```

# --- Work in a disposable directory -----
# Never unpack an untrusted archive next to anything that matters: a malicious
# member named ../../etc/cron.d/x would escape the current directory.
WORKDIR="$(mktemp -d "${TMPDIR:-/tmp}/unwrap.XXXXXXX")" || exit 1
echo "[*] Working directory: $WORKDIR"

cp -- "$INPUT" "$WORKDIR/layer_00"
cd "$WORKDIR" || exit 1

current="layer_00"
layer=0

# --- Main loop -----
# The loop bound is not decoration. If a layer is unrecognised and the file
# never changes, an unbounded loop spins forever. Anything driven by external
# data needs a hard stop.
while (( layer < MAX_LAYERS )); do

    filetype="$(file -b -- "$current")"
    printf "[%02d] %-20s %s\n" "$layer" "$current" "${filetype:0:60}"

    (( layer++ ))
    next="layer_$(printf '%02d' "$layer")"

    case "$filetype" in
        # ---- Terminal condition: plain text is the payload -----
        *"ASCII text"*)
            echo
            echo "[+] Plain text reached after $((layer-1)) layers."
            echo "-----"
            cat -- "$current"
            echo "-----"
            echo "[*] Files left in: $WORKDIR"
            exit 0
            ;;

        # ---- Single-stream compressors -----
        # -c writes to stdout: no extension rename needed, input preserved.
        *"gzip compressed"*)
            gunzip -c -- "$current" > "$next" ;;
        *"bzip2 compressed"*)
            bunzip2 -c -- "$current" > "$next" ;;
        *"XZ compressed"*)
            xz -dc -- "$current" > "$next" ;;
        *"Zstandard"*)
            zstd -dc -- "$current" > "$next" 2>/dev/null ;;

        # ---- Archives: output name is NOT predictable -----
        # tar extracts under the name stored inside the archive, so the file
        # that appears has to be discovered, not assumed.
        *"tar archive"*)
            extract_dir="extract_$layer"
            mkdir -p "$extract_dir"
            tar -xf "$current" -C "$extract_dir" || { echo "[!] tar failed" >&2;
            exit 1; }
    esac
done

```

```

# Expect exactly one member; more than one means this script's
# single-chain assumption doesn't hold for this file.
mapfile -t members < <(find "$extract_dir" -type f)
if (( ${#members[@]} != 1 )); then
    echo "[!] Expected 1 file in archive, found ${#members[@]}:" >&2
    printf '    %s\n' "${members[@]}" >&2
    echo "[!] Inspect manually in $WORKDIR" >&2
    exit 1
fi
mv -- "${members[0]}" "$next"
rmdir "$extract_dir" 2>/dev/null
;;

*)
    echo "[!] Unhandled type: $filetype" >&2
    echo "[!] Stopped at $WORKDIR/$current" >&2
    exit 1
;;

esac

if [[ ! -s "$next" ]]; then
    echo "[!] Layer $layer produced an empty file - aborting." >&2
    exit 1
fi

current="$next"
done

echo "[!] Hit the $MAX_LAYERS layer limit without reaching plain text." >&2
echo "[!] Either the file is deeper than expected, or the loop is stuck." >&2
exit 1

```

Usage

```

./unwrap.sh data.txt          # after xxd -r, on the binary
./unwrap.sh <file> 50         # raise the layer limit

```

Wraps the same commands used manually, so the trace matches what the terminal showed step by step.

Key decisions:

```

gunzip -c -- "$current" > "$next"
bunzip2 -c -- "$current" > "$next"

```

`-c` writes to stdout, which **skips the extension check** and preserves the input. Every `mv ... .gz` from the manual solve disappears. Understanding *why* the rename was needed is what makes it safe to drop.

```

mapfile -t members < <(find "$extract_dir" -type f)
if (( ${#members[@]} != 1 )); then ... fi

```

`tar` output is discovered by listing the extraction directory, never assumed — the exact mistake that cost time manually (`tar -xf next2.tar` when the file was `next2.out`). More than one member means the single-chain assumption doesn't hold, so it stops rather than guessing.

```
WORKDIR="$(mktemp -d "${TMPDIR:-/tmp}/unwrap.XXXXXXX")"
```

Extraction happens in a disposable directory. A malicious archive member named `../../../../etc/cron.d/backdoor` would otherwise write outside it.

```
set -uo pipefail
```

`-u` catches unset variables; `pipefail` makes a pipeline fail if **any** stage fails, not just the last. `-e` is deliberately omitted — the script handles its own error paths and wants control over them.

Trace on the real file:

```
[00] layer_00    gzip compressed data, was "data2.bin"
[01] layer_01    bzip2 compressed data, block size = 900k
[02] layer_02    gzip compressed data, was "data4.bin"
[03] layer_03    POSIX tar archive (GNU)
[04] layer_04    POSIX tar archive (GNU)
[05] layer_05    bzip2 compressed data, block size = 900k
[06] layer_06    POSIX tar archive (GNU)
[07] layer_07    gzip compressed data, was "data9.bin"
[08] layer_08    ASCII text
[+] Plain text reached after 8 layers.
```

Python version — `unwrap.py`

Full source

```
#!/usr/bin/env python3
"""
unwrap.py - Recursively unwrap a repeatedly compressed file.

Written for OverTheWire Bandit Level 12, but works on any nested archive.

Unlike the Bash version, this calls no external commands. It reads magic
numbers directly and uses the standard library's gzip, bz2, lzma and tarfile
modules, so the whole chain is handled in memory.

Usage:
    ./unwrap.py <file> [--max-layers N] [--verbose]
    ./unwrap.py data.txt --hexdump          # input is an xxd-style hex dump
"""

import argparse
import bz2
import gzip
```

```

import io
import lzma
import re
import sys
import tarfile

MAX_MEMBER_SIZE = 100 * 1024 * 1024 # 100 MB – decompression-bomb guard


def revert_hexdump(text: str) -> bytes:
    """Reverse an xxd-style hex dump back into raw bytes.

    Format: OFFSET: HH HH HH HH  ASCII
    The ASCII column must be discarded, and it can itself contain characters
    that look like hex, so slicing by column is unreliable. Splitting on the
    colon and then keeping only well-formed hex groups is robust.
    """
    out = bytearray()
    for line in text.splitlines():
        if ":" not in line:
            continue
        body = line.split(":", 1)[1]
        # Hex groups are 1-4 hex chars; the ASCII column is separated by 2+ spaces
        hex_part = re.split(r"\s{2,}", body.strip())[0]
        for group in hex_part.split():
            if re.fullmatch(r"[0-9a-fA-F]+", group) and len(group) % 2 == 0:
                out += bytes.fromhex(group)
    return bytes(out)


def identify(data: bytes) -> str:
    """Identify a format by its magic number.

    This is what `file` does: read the leading bytes, ignore the filename.
    A file's extension is a human convention and guarantees nothing.
    """
    if data[:2] == b"\x1f\x8b":
        return "gzip"
    if data[:3] == b"BZh":
        return "bzip2"
    if data[:6] == b"\xfd7zXZ\x00":
        return "xz"
    if data[257:262] == b"ustar": # tar's magic sits at offset 257
        return "tar"
    if is_text(data):
        return "text"
    return "unknown"


def is_text(data: bytes) -> bool:
    """Heuristic: no null bytes and decodable as UTF-8."""
    if b"\x00" in data[:8192]:
        return False

```

```

try:
    data[:8192].decode("utf-8")
    return True
except UnicodeDecodeError:
    return False

```

```

def untar(data: bytes) -> bytes:
    """Extract a single-member tar archive, in memory.

    Never uses extractall(): a member named ../../etc/cron.d/x would write
    outside the target directory (Zip Slip / CVE-2007-4559). Reading the
    member object directly sidesteps the filesystem entirely.
    """
    with tarfile.open(fileobj=io.BytesIO(data)) as tar:
        members = [m for m in tar.getmembers() if m.isfile()]
        if len(members) != 1:
            raise ValueError(
                f"expected 1 file in archive, found {len(members)}: "
                f"{[m.name for m in members]}"
            )
        member = members[0]
        if member.size > MAX_MEMBER_SIZE:
            raise ValueError(f"member is {member.size} bytes - refusing (bomb
guard)")
        extracted = tar.extractfile(member)
        if extracted is None:
            raise ValueError(f"could not read member {member.name}")
        return extracted.read()

```

```

DECOMPRESSORS = {
    "gzip": gzip.decompress,
    "bzip2": bz2.decompress,
    "xz": lzma.decompress,
    "tar": untar,
}

```

```

def unwrap(data: bytes, max_layers: int = 30, verbose: bool = True) -> bytes:
    """Peel compression layers until plain text is reached.

    The layer cap is essential: if a format is misidentified and the data
    stops changing, an unbounded loop never terminates.
    """
    for layer in range(max_layers):
        kind = identify(data)

        if verbose:
            print(f"[{layer:02d}] {kind:<10} {len(data):>10}, {bytes}",
file=sys.stderr)

        if kind == "text":

```

```

        return data
    if kind == "unknown":
        raise ValueError(
            f"unrecognised format at layer {layer}; "
            f"first bytes: {data[:16].hex(' ')}"
        )

    try:
        data = DECOMPRESSORS[kind](data)
    except Exception as exc:
        raise ValueError(f"failed to decompress {kind} at layer {layer}: {exc}")
from exc

    raise ValueError(f"hit the {max_layers}-layer limit without reaching text")

def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__,
    formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument("file", help="file to unwrap")
    parser.add_argument("--hexdump", action="store_true",
                        help="treat the input as an xxd-style hex dump")
    parser.add_argument("--max-layers", type=int, default=30)
    parser.add_argument("-q", "--quiet", action="store_true",
                        help="suppress the per-layer trace")
    args = parser.parse_args()

    try:
        with open(args.file, "rb") as fh:
            data = fh.read()
    except OSError as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 1

    # Auto-detect a hex dump so --hexdump is usually unnecessary
    if args.hexdump or re.match(rb"^[0-9a-fA-F]{8}: ", data[:10]):
        if not args.quiet:
            print("[*] Input looks like a hex dump - reverting", file=sys.stderr)
            data = revert_hexdump(data.decode("ascii", errors="replace"))

    try:
        result = unwrap(data, args.max_layers, verbose=not args.quiet)
    except ValueError as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 1

    if not args.quiet:
        print("-" * 40, file=sys.stderr)
    sys.stdout.write(result.decode("utf-8", errors="replace"))
    return 0

```


Never assume an interpreter is installed on a machine you don't administer. On a locked-down host, the Bash version may be the only option.

Step 1 — A writable working directory

The Bandit home is read-only, so everything happens in `/tmp` under a hard-to-guess name (the directory is shared with every other player):

```
mkdir -p /tmp/dk9x2mqf7t && cd /tmp/dk9x2mqf7t
cp ~/data.txt .
```

Step 2 — Get the script onto the box

Option A — paste with a heredoc. Works over any SSH session, needs nothing installed:

```
cat > unwrap.sh << 'ENDOFSCRIPT'
#!/usr/bin/env bash
... paste the whole script here ...
ENDOFSCRIPT
```

⚠ **The quotes around 'ENDOFSCRIPT' are mandatory.** Unquoted, bash performs variable expansion *while reading the heredoc* — every `$1`, `$WORKDIR` and `${#members[@]}` in the script would be replaced by its (empty) value as it's being written, silently producing a broken file. Quoting the delimiter makes the block literal. This is the single most common way a pasted script arrives corrupted.

Option B — scp from the local machine:

```
scp -P 2220 unwrap.sh bandit12@bandit.labs.overthewire.org:/tmp/dk9x2mqf7t/
```

Note the **capital** `-P`: `scp` uses `-P` for the port while `ssh` uses lowercase `-p`. In `scp`, lowercase `-p` means "preserve timestamps" — a mismatch that has confused people for thirty years.

Step 3 — Fix line endings if the file came from Windows

```
head -1 unwrap.sh | cat -A | tail -c 20
```

If the line ends in `^M$`, the file has Windows CRLF endings and the shebang will fail with the notoriously unhelpful `bad interpreter: /usr/bin/env bash^M: No such file or directory` — the `^M` is part of the path bash is trying to find.

```
tr -d '\r' < unwrap.sh > unwrap_fixed.sh && mv unwrap_fixed.sh unwrap.sh
```

The `tr -d '\r'` from [Linux - Encoding vs Encryption](#), solving a real problem. Note the redirect goes to a **different filename** — `tr -d '\r' < unwrap.sh > unwrap.sh` would truncate the file before reading it, the exact mistake from [Bandit - Level 12](#).

Step 4 — Make it executable

```
chmod +x unwrap.sh
```

Without the execute bit, `./unwrap.sh` returns `Permission denied` — see [Linux - Permissions & Process Management](#). Alternatively `bash unwrap.sh` runs it without the bit, since the interpreter is invoked explicitly.

Step 5 — Run it

Bash version — revert the hex dump first:

```
xxd -r data.txt > start.bin  
./unwrap.sh start.bin
```

⚠ **Do not pass `data.txt` directly to the Bash version.** `file` reports the hex dump as `ASCII text`, so the script's terminal condition matches on layer 0 and it triumphantly prints the hex dump as though it were the answer. Not a crash — a **wrong answer delivered confidently**, the most dangerous kind of bug and the same failure shape as `base64` without `-d` in [Bandit - Level 10](#).

Python version — no pre-step needed:

```
python3 unwrap.py data.txt
```

It detects the `OFFSET: HH HH` pattern and reverts the dump itself.

Step 6 — Expected output

```
[*] Working directory: /tmp/unwrap.63aRvzbb  
[00] layer_00    gzip compressed data, was "data2.bin"  
[01] layer_01    bzip2 compressed data, block size = 900k  
[02] layer_02    gzip compressed data, was "data4.bin"  
[03] layer_03    POSIX tar archive (GNU)  
[04] layer_04    POSIX tar archive (GNU)  
[05] layer_05    bzip2 compressed data, block size = 900k  
[06] layer_06    POSIX tar archive (GNU)  
[07] layer_07    gzip compressed data, was "data9.bin"  
[08] layer_08    ASCII text
```

```
[+] Plain text reached after 8 layers.
```

```
-----  
The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk  
-----
```

The trace should match the manual solve layer for layer. If it doesn't, the script is wrong — **the manual result is the reference**, not the other way round.

Result

```

Windows PowerShell
bandit12@bandit: /tmp/dk9x2mqf7t

--[ More information ]--

For more information regarding individual wargames, visit
http://www.overthewire.org/wargames/

For support, questions or comments, contact us on discord or IRC.

Enjoy your stay!

bandit12@bandit:~$ ls
data.txt
bandit12@bandit:~$ which xxd file gunzip bunzip2 tar
/usr/bin/xxd
/usr/bin/file
/usr/bin/gunzip
/usr/bin/bunzip2
/usr/bin/tar
bandit12@bandit:~$ which python3
/usr/bin/python3
bandit12@bandit:~$ python3 --version
Python 3.14.4
bandit12@bandit:~$ mkdir -p /tmp/dk9x2mqf7t && cd /tmp/dk9x2mqf7t
cp ~/data.txt .
bandit12@bandit:/tmp/dk9x2mqf7t$ ls
data.txt
bandit12@bandit:/tmp/dk9x2mqf7t$ cat > unwrap.sh
<<
#!/usr/bin/env bash
^C
bandit12@bandit:/tmp/dk9x2mqf7t$ ls
data.txt  unwrap.sh
bandit12@bandit:/tmp/dk9x2mqf7t$ nano unwrap.sh
Unable to create directory /home/bandit12/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

bandit12@bandit:/tmp/dk9x2mqf7t$ ls -la
total 8
drwxrwxr-x  2 bandit12 bandit12   80 Aug 19 14:27 .
drwxrwx-wt 120 root      root      3140 Aug 19 14:28 ..
-rw-r----- 1 bandit12 bandit12  2641 Aug 19 14:24 data.txt
-rw-rw-r--  1 bandit12 bandit12  3654 Aug 19 14:27 unwrap.sh
bandit12@bandit:/tmp/dk9x2mqf7t$ chmod +x unwrap.sh
bandit12@bandit:/tmp/dk9x2mqf7t$ ls -la
total 8
drwxrwxr-x  2 bandit12 bandit12   80 Aug 19 14:27 .
drwxrwx-wt 120 root      root      3160 Aug 19 14:28 ..
-rw-r----- 1 bandit12 bandit12  2641 Aug 19 14:24 data.txt
-rwxrwxr-x  1 bandit12 bandit12  3654 Aug 19 14:27 unwrap.sh
bandit12@bandit:/tmp/dk9x2mqf7t$ xxd -r data.txt > start.bin
bandit12@bandit:/tmp/dk9x2mqf7t$ ls
data.txt  start.bin  unwrap.sh
bandit12@bandit:/tmp/dk9x2mqf7t$ ./unwrap.sh start.bin
[*] Working directory: /tmp/unwrap.5pk7zR64
[00] layer_00      gzip compressed data, was "data2.bin", last modified: Wed Ju
[01] layer_01      bzip2 compressed data, block size = 900k
[02] layer_02      gzip compressed data, was "data4.bin", last modified: Wed Ju
[03] layer_03      POSIX tar archive (GNU)
[04] layer_04      POSIX tar archive (GNU)
[05] layer_05      bzip2 compressed data, block size = 900k
[06] layer_06      POSIX tar archive (GNU)
[07] layer_07      gzip compressed data, was "data9.bin", last modified: Wed Ju
[08] layer_08      ASCII text

[+] Plain text reached after 8 layers.
-----
The password is qQYQihOBPR8zR6lqxYqX45quvihF2uzk
-----
[*] Files left in: /tmp/unwrap.5pk7zR64
bandit12@bandit:/tmp/dk9x2mqf7t$

```

```
--[ More information ]--

For more information regarding individual wargames, visit
http://www.overthewire.org/wargames/

For support, questions or comments, contact us on discord or IRC.

Enjoy your stay!

bandit12@bandit:~$ ls
data.txt
bandit12@bandit:~$ mkdir -p /tmp/david && cd /tmp/david
bandit12@bandit:/tmp/david$ cp ~/data.txt .
bandit12@bandit:/tmp/david$ ls
data.txt
bandit12@bandit:/tmp/david$ nano unwrap.py
Unable to create directory /home/bandit12/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

bandit12@bandit:/tmp/david$ chmod +x unwrap.py
bandit12@bandit:/tmp/david$ ls -la
total 12
drwxrwxr-x  2 bandit12 bandit12  80 Aug 19 14:34 .
drwxrwx-wt 108 root      root    2780 Aug 19 14:35 ..
-rw-r----- 1 bandit12 bandit12 2641 Aug 19 14:33 data.txt
-rwxrwxr-x  1 bandit12 bandit12 5785 Aug 19 14:34 unwrap.py
bandit12@bandit:/tmp/david$ xxd -r data.txt > solution1.bin
bandit12@bandit:/tmp/david$ ls
data.txt  solution1.bin  unwrap.py
bandit12@bandit:/tmp/david$ ./unwrap.py solution1.bin
[00] gzip          613 bytes
[01] bzip2          580 bytes
[02] gzip          438 bytes
[03] tar           20,480 bytes
[04] tar           10,240 bytes
[05] bzip2          223 bytes
[06] tar           10,240 bytes
[07] gzip           79 bytes
[08] text           49 bytes
-----
The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk
bandit12@bandit:/tmp/david$
```

Troubleshooting

Symptom	Cause	Fix
bad interpreter: ...^M	Windows CRLF endings	<code>tr -d '\r'</code> (Step 3)
Permission denied running <code>./unwrap.sh</code>	No execute bit	<code>chmod +x</code> , or run <code>bash unwrap.sh</code>
Prints the hex dump as "plain text"	Passed <code>data.txt</code> to the Bash version	<code>xxd -r first</code>
Unhandled type: ...	Format outside the four supported	Add a <code>case</code> branch / dict entry
Empty or garbled script after pasting	Unquoted heredoc delimiter	Re-paste with <code><< 'EOF'</code>
command not found: <code>python3</code>	No interpreter on the host	Use the Bash version
<code>mkdir</code> : cannot create directory	Wrong path, or read-only location	Work under <code>/tmp</code>

Cleaning up

The level banner asks players not to leave files lying around:

```
cd ~ && rm -rf /tmp/dk9x2mqf7t /tmp/unwrap.*
```

Good habit beyond etiquette: scratch directories holding decompressed unknown content shouldn't outlive the task that created them.

Comparison

	Bash	Python
Dependencies	<code>file</code> , <code>gzip</code> , <code>bzip2</code> , <code>tar</code> , <code>xxd</code>	standard library only
Where data lives	temp files on disk	memory
Format detection	delegated to <code>file</code>	own magic-number table
Hex dump	needs <code>xxd -r</code> first	built in, auto-detected
Best for	quick work on a box you just SSH'd into	repeatable tooling, integration

Bash wins when you're already in a shell on an unfamiliar host with no Python available. **Python wins** for anything reused, tested, or integrated — better error handling, no subprocess overhead, no temp files, and precise control over the security guards.

The general rule: **shell for gluing existing tools, a real language once there's state, error handling or logic to manage**. This task sits right on that boundary, which is why it's worth writing both.

Limitations

- Assumes a **single chain** — one file per archive. Both stop with a clear error otherwise.
 - Bash version needs `xxd -r` run first; Python handles the hex dump itself.
 - No support for `zip`, `7z`, `rar` or `cpio`. Adding a format is one `case` branch or one dict entry.
 - The Python bomb guard checks tar member sizes only; a gzip bomb would still decompress into memory. A production version would stream with a size cap.
-

What This Exercise Teaches Beyond Bandit

The *inspect* → *branch* → *act* → *re-inspect* loop is the shape of real unpacking work: a malware delivery chain (`.eml` → `.zip` → `.iso` → `.lnk`), a firmware image, a set of rotated logs. `binwalk` is essentially this loop generalised — scanning for every known magic number at every offset.

And the reflex worth keeping from writing it: **automate second**. Doing it by hand first is what surfaced the three awkward cases — unpredictable output names, inconsistent input consumption, and the need for a loop bound. None of them were visible from the problem statement, and a script written before the manual solve would have hit all three as mysterious bugs.